NLP Research Assignment

Hallucination Detection in Retrieval-Augmented Language Models

Track A: Dynamic Uncertainty-Aware Attribution

CS F429 — Natural Language Processing
BITS Pilani, Dubai Campus | Semester II 2025-26

Team: Zayaan, Ridhwan, Aaqib, Mariah

This document provides a complete explanation of the project —
what it does, how it works, and how it maps to the evaluation rubric.
Intended for team members to understand and present the work confidently.

AUROC	Datasets	Models	Experiments	Marks Target
0.734	2	6 LLMs	8	12-13 / 15

1. What Is This Project?

The Problem We Are Solving

Modern AI systems like ChatGPT, Llama, and Mistral are often used in a setup called **Retrieval-Augmented Generation (RAG)**. In RAG, when you ask the AI a question, it first retrieves some background information (called context) from a database, and then uses that context to generate an answer.

The problem is that even with this retrieved context, AI models sometimes make up information that is not supported by the context. This is called **hallucination**. For example, if you ask "When did Anne Frank die?" and the context says "February 1945", the AI might still answer "February 7, 2022" — fabricating a wrong date.

> **Our project builds a system that automatically detects when an AI is hallucinating.**

Our Core Idea — In Simple Terms

Think of it this way: if you give someone a fact and they already knew it, their confidence should increase. If they make something up, giving them the real fact does not change their confidence — because they are not using it.

We apply the same logic to AI. We run the model twice:

Step	What We Do	What We Measure
1	Run AI WITHOUT context	How uncertain is it at each word?
2	Run AI WITH context	How uncertain is it now?
3	Compare both runs	Did context reduce uncertainty?
4	Score the answer	Low reduction = likely hallucination

If the AI is hallucinating, context does not help it — so its uncertainty stays high. If the AI is being faithful, context confirms what it knows — uncertainty drops. We measure this change and use it as a hallucination signal.

2. How Does It Work? — The Technical Details

2.1 The Base Model — GPT-2

We use **GPT-2** as our base language model. GPT-2 is a transformer model that, given any text input, predicts a probability distribution over all possible next tokens (words/subwords). This gives us access to the model's internal uncertainty at each position.

For every token the model generates, we get a vector of 50,257 probabilities — one for each word in GPT-2's vocabulary. These probabilities are what we analyse.

2.2 The Five Metrics We Compute

Metric 1: Shannon Entropy (Entropy Only — Baseline)

Shannon Entropy measures how uncertain the model is about the next token. If the model is very confident (assigns high probability to one token), entropy is LOW. If the model is confused (spreads probability across many tokens), entropy is HIGH.

Mathematical Formula:

$$H(p) = -SUM [p(x) * log(p(x))] for all x in vocabulary$$

Where p(x) is the probability the model assigns to token x. We compute this for every token position in the answer and take the average.

Code:

```
from scipy.stats import entropy entropy_score = entropy(token_probabilities) # one value per token
```

Metric 2: Information Gain (IG) — Our Main Metric

Information Gain measures how much the entropy DROPS when we add the retrieved context. A high IG means the context helped the model — faithful answer. A low or negative IG means the context did not help — possible hallucination.

Mathematical Formula:

$$IG(t) = H(p\_no\_context)(t) - H(p\_with\_context)(t)$$

Where t is the token position, H is entropy, p_no_context is the probability distribution without context, and p_with_context is the distribution with context.

Code:

```
entropy_no_context = compute_entropy(probs_no_ctx) # run without context
entropy_with_context = compute_entropy(probs_with_ctx) # run with context ig_per_token = entropy_no_context - entropy_with_context information_gain = float(np.mean(ig_per_token)) # average across all tokens
```

Metric 3: KL Divergence — Our Strongest Metric

KL Divergence (Kullback-Leibler Divergence) measures how DIFFERENT the entire probability distribution is between the no-context run and the with-context run. Unlike IG which only looks at entropy, KL

divergence captures the full shape of the distribution change. This turned out to be our strongest individual metric (AUROC 0.723).

Mathematical Formula:

$$KL(P \,||\, Q) = SUM\ [\ P(x) * log(\ P(x) / Q(x)\ )\ ]\ for\ all\ x$$

Where P is the distribution WITHOUT context and Q is the distribution WITH context. A high KL means context shifted the model significantly — this can signal hallucination when the shift is inconsistent with the context content.

> This is the metric the evaluator will specifically ask about. Be ready to explain it!

Code (the evaluator will ask you to show this):

```
from scipy.special import kl_div kl_per_token = [] for p_no, p_with in zip(probs_no_ctx, probs_with_ctx): # Add small value to avoid log(0) p_no = p_no + 1e-10 p_no = p_no / p_no.sum() # normalize to sum to 1 p_with = p_with + 1e-10 p_with = p_with / p_with.sum() # normalize # kl_div computes element-wise KL, sum gives total KL for this token kl = float(np.sum(kl_div(p_no, p_with))) kl_per_token.append(kl) kl_divergence = float(np.mean(kl_per_token)) # average across tokens
```

Metric 4: Confidence Drop

Confidence Drop measures how the model's MAXIMUM token probability changes when context is added. The maximum probability represents the model's confidence in its top choice. If this drops significantly with context, it signals the context contradicted the model.

Mathematical Formula:

$$ConfDrop(t) = max(p\_no\_context(t)) - max(p\_with\_context(t))$$

Code:

```
conf_no_ctx = np.max(probs_no_ctx, axis=1) # max prob at each token, no context
conf_with_ctx = np.max(probs_with_ctx, axis=1) # max prob at each token, with context
confidence_drop = float(np.mean(conf_no_ctx - conf_with_ctx))
```

Metric 5: Semantic Entropy

Semantic Entropy computes entropy over only the TOP-10 most probable tokens instead of all 50,257. This focuses on meaning-level uncertainty — is the model confused between semantically similar options? This is a simplified version of the full semantic entropy concept.

Code:

```
sem_entropies = [] for token_probs in probs_with_ctx: top10_idx = np.argsort(token_probs)[-10:] # get top 10 token indices top10_probs = token_probs[top10_idx] # their probabilities top10_probs = top10_probs / top10_probs.sum() # normalize sem_h = entropy(top10_probs) # entropy over top 10 only sem_entropies.append(sem_h) semantic_entropy = float(np.mean(sem_entropies))
```

2.3 The Composite Score

Instead of using just one metric, we combine all four strongest metrics into a single **composite hallucination score** using Logistic Regression — a simple but effective machine learning model. We trained the logistic regression on 300 samples from the RAGTruth training split and evaluated on 500 samples from the test split.

Code:

```
from sklearn.linear_model import LogisticRegression from sklearn.preprocessing import StandardScaler # Features used: entropy, IG, KL divergence, confidence drop feature_cols = ["entropy_only", "information_gain", "kl_divergence", "confidence_drop"] # Scale features (important for logistic regression) scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # Train composite lr = LogisticRegression(random_state=42, max_iter=1000, C=0.5) lr.fit(X_train_scaled, y_train) # Get hallucination probability for each sample composite_score = lr.predict_proba(X_test_scaled)[:, 1]
```

2.4 The Two Datasets We Used

Dataset	Size Used	Labels	Purpose
RAGTruth (wandb/RAGTruth-processed)	500 test samples 300 train samples	Contradictory Unsupported	Main experiments (Exp 1,2,3,5,6,7,8)
HaluEval (pminervini/HaluEval)	200 samples	Yes / No	Cross-domain transfer (Experiment 4)

3. Experiments and Results

We ran 8 experiments as required by the rubric. Here are the results with explanation:

Experiments 1 & 2 — Composite AUROC on RAGTruth (4 marks)

What is AUROC? AUROC (Area Under the ROC Curve) measures how well our system separates hallucinated from non-hallucinated samples. A score of 1.0 is perfect, 0.5 is random guessing. We need ≥ 0.75 for full marks.

Metric / Composite	AUROC	95% CI	F1	Spearman	ECE
Entropy only (Baseline 1)	0.5806	[0.532-0.633]	0.6800	0.1396	0.1054
SelfCheckGPT (Baseline 2)	0.6805	[0.630-0.729]	0.6714	0.3127	0.0873
+ Information Gain	0.2963	[0.251-0.342]	0.6029	-0.3529	0.3279
+ KL divergence	0.7235	[0.678-0.769]	0.7000	0.3871	0.1235
+ Confidence drop	0.2859	[0.245-0.334]	0.6057	-0.3708	0.3614
+ Semantic entropy	0.2734	[0.227-0.322]	0.6086	-0.3924	0.3830
Full composite	0.7338	[0.688-0.776]	0.6944	0.4049	0.1178

n=500 (250 hallucinated, 250 clean) from RAGTruth test split. Bootstrap CI: 1000 iterations.

Our composite (0.734) beats both baselines — Entropy only (0.581) and SelfCheckGPT (0.681). We are in the 3-mark range (0.68-0.74). KL divergence is our strongest individual metric.

Experiment 3 — Temporal Precedence (2 marks)

This experiment checks whether our metrics show a signal BEFORE the hallucination happens. We use real character-level span positions from RAGTruth annotations to find exactly which token position the hallucination starts (called t), then measure metrics at t-3 to t+1.

Position	IG	KL Divergence	Predictive H	Conf Drop
t-3	0.6505	10.7348	2.6657	-0.0976
t-2	0.0385	9.1889	2.8544	-0.0768
t-1	1.5087	9.2260	2.8072	-0.2390
t (onset)	1.1104	10.3816	2.5171	-0.2042
t+1	0.6219	10.9170	2.5651	-0.1366

n=141 samples with real span positions. Mann-Whitney U test reported (p=0.11, p=0.64).

KL divergence shows elevated signal at t-3 (10.73) suggesting pre-hallucination uncertainty. Mann-Whitney test not significant — we earn 1/2 marks here. Line plot saved to outputs/experiment3_temporal_real.png.

Experiment 4 — Cross-Domain Transfer to HaluEval (2 marks)

We run our composite on HaluEval with ZERO retraining or recalibration — pure zero-shot transfer. This tests whether our metric generalizes to new domains and datasets.

Metric	AUROC RAGTruth	AUROC HaluEval	Drop	Rank Stable?
Full composite	0.7338	0.9249	-0.2006 (improved)	Yes
Information Gain	0.3482	0.4908	-0.1426 (improved)	Yes
KL divergence	0.7235	0.9060	-0.2086 (improved)	Yes

Zero-shot transfer — no retraining or recalibration applied. n=200 HaluEval samples.

Our metrics actually IMPROVED on HaluEval (0.925 vs 0.734). This is a strong result — it shows our uncertainty-based approach generalizes well. Full 2/2 marks.

Experiment 5 — Hallucination Type Breakdown (2 marks)

RAGTruth labels hallucinations as either **contradictory** (conflicts with context) or **unsupported** (not mentioned in context). We report AUROC separately for each type.

Hallucination Type	Count in Test Set	Composite AUROC	Best Single Metric
Contradictory	116	0.7841	KL divergence (0.7795)
Unsupported	134	0.6902	KL divergence (0.6750)

AUROC gap: 0.0939 (just below 0.10 threshold — 1/2 marks).

Finding: Contradictory hallucinations are easier to detect (0.784) than unsupported ones (0.690). This matches the expected finding — when an AI contradicts the context, the uncertainty signal is stronger than when it simply omits information.

Experiment 6 — AUROC by Generator Model (Part of Exp 6-8, 2 marks)

Model	Count	AUROC	Interpretation
gpt-3.5-turbo-0613	80	0.8185	Easiest to detect
mistral-7B-instruct	79	0.8106	Strong signal
llama-2-13b-chat	107	0.8005	Strong signal
llama-2-70b-chat	85	0.7970	Good signal
gpt-4-0613	66	0.7176	Moderate signal
llama-2-7b-chat	83	0.6298	Weakest signal

Key finding: Our metric works best on GPT-3.5 and Mistral, and least well on Llama-2-7B. Smaller models (7B) may produce less distinctive probability distributions, making uncertainty signals harder to interpret. This is a genuine research finding.

Experiment 8 — SOTA Gap Analysis

Method	AUROC	Type
Entropy Only (our baseline)	0.5806	Our baseline
SelfCheckGPT	0.6805	Published baseline
Our Full Composite	**0.7338**	**Our method**
ReDeEP	0.8200	SOTA
LUMINA	0.8700	SOTA

SOTA gap closed: 52.9% — above the 50% threshold for full marks. Formula: (0.734 - 0.581) / (0.870 - 0.581) x 100 = 52.9%

4. Rubric Alignment — How We Map to Each Mark

Component	Max	Our Score	Condition Met	Evidence
Exp 1&2 Composite AUROC on RAGTruth	4	3	AUROC 0.68-0.74 both baselines beaten	Composite 0.734 Beats entropy (0.581) Beats SelfCheck (0.681)
Exp 3 Temporal Precedence	2	1	Best metric peaks at t-1; plot included	KL elevated at t-3 Plot generated Mann-Whitney p=0.11
Exp 4 Cross-domain Transfer	2	2	HaluEval AUROC >=0.68 AND instability explained	HaluEval 0.925 Rank stable across both datasets
Exp 5 Type Breakdown	2	1	AUROC gap >0.10 with explanation	Gap = 0.0939 Contradictory 0.784 Unsupported 0.690
Exp 6-8 Deeper Analysis	2	2	All three attempted SOTA gap >=50%	Generator table done 3 failure cases 52.9% gap closed
Code & Demo Live Pipeline	3	3	Runs on unseen input; member explains code	05_demo.ipynb ready All members briefed on KL divergence
TOTAL	**15**	**12**		

5. Team Roles and Responsibilities

Team Member	Role	Task	Status
Zayaan	Lead Developer	All code, experiments, result tables	Done
Ridhwan	Presentation	PowerPoint slides with final numbers	In Progress
Aaqib	Documentation	README file Setup instructions	Draft Ready
Mariah	Literature	Paper summaries for references slide	In Progress

6. How to Explain the Project to Ma'am

Suggested Opening (30 seconds)

"We built an unsupervised hallucination detector for RAG systems. The core idea is that a faithful answer should show reduced uncertainty when retrieved context is provided. We quantify this reduction using four token-level metrics — entropy, information gain, KL divergence, and confidence drop — and combine them into a logistic regression composite that achieves 0.734 AUROC on the RAGTruth test set, beating both baselines."

If Asked About KL Divergence Specifically

"KL divergence measures how different the model's token probability distribution is when context is added versus when it is not. For each token position, we compute KL(P_no_context || P_with_context) using scipy's kl_div function, sum across the vocabulary to get the total KL for that token, and average across all token positions. The code is in the compute_all_metrics function in notebook 02_core_pipeline."

Key Numbers to Remember

Metric	Value	What It Means
Composite AUROC	0.734	3/4 marks — beats both baselines
HaluEval AUROC	0.925	Full 2/2 marks — strong transfer
SOTA gap closed	52.9%	Full 2/2 marks — above 50% threshold
Type gap	0.0939	1/2 marks — just below 0.10
Test samples	500	Professor approved this size

Train samples	300	Separate split — no data leakage

7. How to Run the Code (For Teammates)

Quick Start — Demo Only

If you just want to test the pipeline:

```
cd C:\Nlp_Assignment nlp_env\Scripts\activate jupyter notebook
```

Then open 05_demo.ipynb and run all cells.

Notebook Run Order

Notebook	Purpose	Runtime
01_data_exploration.ipynb	Load RAGTruth and HaluEval datasets	5 min
02_core_pipeline.ipynb	Compute all 5 metrics on test data	20 min
03_baselines.ipynb	Run SelfCheckGPT baseline	15 min
04_experiments.ipynb	Run all 8 experiments	30 min
05_demo.ipynb	Live demo for evaluator	2 min